
M I R O I R

Bloc 3

Pipelines de données temps réel

Industrialisation d'un flux transactionnel pour le moteur de recommandation Miroir.

Prince Néhémie · École des Mines · Juin 2026

github.com/princebn/miroir

C O N T E X T E

Miroir : SaaS B2B de conseil en image

Le Bloc 3 alimente en signaux temps réel le moteur de recommandation conçu au Bloc 4.

Persona — Aïcha

- Conseillère indépendante
- ~30 clientes premium (Europe + Afrique)
- Profil pro : haute rotation, exigence forte

Pain principal

- 60 % du temps en recherche produit
- Le catalogue scale, la conseillère ne scale pas

Valeur Miroir

- Recos personnalisées validées par la conseillère
- ~10h/semaine gagnées par conseillère

Bloc 3 dans le tout

- Pipeline temps réel = signaux de demande
- Alimente la reco (Bloc 4) avec données fraîches

Objectif & compétences évaluées

5 compétences RNCP — toutes couvertes par les livrables de ce bloc.

1

Concevoir un système de gestion de données temps réel

Kafka KRaft, topic miroir.transactions, partitions.

2

Établir un pipeline ETL/ELT

Producer Python → Consumer Postgres → dbt (staging + marts).

3

Automatiser les flux

DAG Airflow miroir_pipeline (4 tâches, exclude tag:reference).

4

Surveiller pour qualité & gouvernance

dbt tests + Great Expectations (6 attentes) + console live.

5

Contrôle qualité & correction d'erreurs

Code de sortie GE propagé : la qualité bloque le DAG.

ARCHITECTURE

Pipeline de bout en bout

Six composants enchaînés, chacun versionné et reproductible.



Producteur — rejeu des transactions H&M

- 31,8 M transactions source (Kaggle)
- Échantillon stratifié par stride sur 2 ans
- Clé Kafka = customer_id (partitionnement stable)

Aval — transform + qualité + monitoring

- dbt : staging (vue) + marts (tables)
- GE : 6 attentes — domaine, non-nullité, bornes
- Console Streamlit + Altair → analyse en direct

Technologies & versions

Versions verrouillées, choix de stabilité (cf. ADR du repo).

STREAMING	Apache Kafka 3.8.0 (KRaft, sans Zookeeper)
OLTP & STOCKAGE	PostgreSQL 16 + pgvector — réutilisé du Bloc 2
TRANSFORMATION	dbt-core 1.11 + dbt-postgres 1.10
QUALITÉ	Great Expectations 1.18 (API 1.x : data sources, validations)
ORCHESTRATION	Apache Airflow 2.10.5 (venv séparé pour isoler les deps)
MONITORING	Streamlit 1.x + Altair 5 (charts Vega-Lite natifs)
VERSIONING	GitHub (princebn/miroir) — code, modèles dbt, slides

Kafka : rejouer les vraies transactions H&M

Producer Python (confluent-kafka) → topic miroir.transactions → consumer Postgres.

Principes

- Kafka 3.8 en mode KRaft (sans Zookeeper)
- Topic : miroir.transactions · 3 partitions
- Clé message = customer_id (cohérence partition)
- Producer ligne par ligne (line-based, sans pandas)
- --stride N : 1 ligne sur N pour étaler sur 2 ans
- Consumer batch 500, idle-stop 5 s

commande type

```
python produce_transactions.py \  
    --rate 0 --limit 4500 --stride 7000
```

```
# 4 500 messages étalés  
# de sept. 2018 à sept. 2020
```

Deux familles de modèles dbt, séparées par tag

Le streaming reste rapide ; les agrégats lourds sur 31,8 M sont construits explicitement.

Stream — tag: stream

- stg_stream_transactions (vue)
- mart_channel_daily (table)
- Tests dbt : not_null, accepted_values {1,2}
- Construit à chaque run du DAG (~secondes)
- Alimente le panneau « flux temps réel »

Reference — tag: reference

- mart_totals · mart_segment
- mart_seg_category · mart_seg_colour
- mart_seg_age (jointure 3-way : tx × articles × customers)
- mart_seg_month — saisonnalité par segment
- Construits une fois sur 31,8 M (~3-7 min)
- Alimentent la console business + heatmap

DAG : **dbt build --exclude tag:reference** → la boucle pipeline reste rapide, les marts lourds restent explicites.

Garde-fous en couches

Trois lignes de défense : tests de modèle, attentes statistiques, garde DAG.

1. dbt tests

- not_null sur les clés
- accepted_values : canal $\in \{1, 2\}$
- Tests exécutés à chaque build
- Échec dbt → échec du DAG

2. Great Expectations

- 6 attentes sur le batch en cours
- Non-nullité customer_id, article_id, t_dat
- Domaine canal $\in \{1, 2\}$
- Bornes prix $\in [0, 1]$ (unité normalisée)
- Volume minimum (≥ 1 ligne)

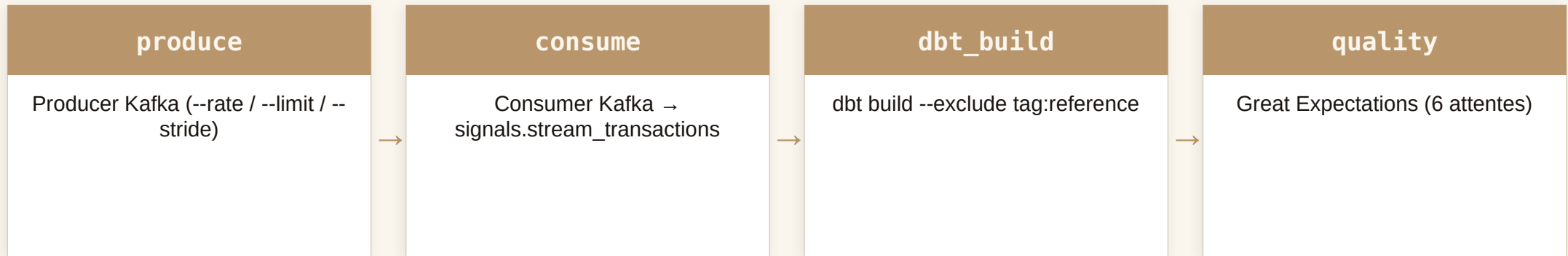
3. Garde DAG

- sys.exit(0|1) selon le succès GE
- BashOperator propage le code de sortie
- Tâche quality échoue → DAG en erreur
- Pas de chargement silencieux de données invalides

Principe : la qualité n'est pas un rapport en aval, c'est une condition d'exécution.

DAG miroir_pipeline

Quatre tâches en série, exécutées par BashOperator dans des venvs isolés.



Validation

- airflow dags test miroir_pipeline 2026-06-06 → 4 tâches SUCCESS
- venv .venv-airflow séparé du .venv data (Polars, dbt, GE) pour isoler les conflits de deps
- Pas d'exemples (LOAD_EXAMPLES=False) — DAG folder pointe sur bloc3-pipelines/dags

D O N N É E S

31,8 M transactions, 2 ans — chaque chiffre mesuré

Source : signals.hm_transactions (Bloc 2). Vérifié via explore_data.py et requêtes Postgres.

TRANSACTIONS

31,79 M

[MESURÉ]

CLIENTS DISTINCTS

1,36 M

[MESURÉ]

ARTICLES DISTINCTS

105 k

[MESURÉ]

PÉRIODE

2 ans

2018-09 → 2020-09

Saisonnalité — pics observés

- Juin 2019 : 1,9 M transactions (pic absolu)
- Juin 2020 : 1,76 M (rebond post-confinement)
- Creux : décembre 2019, février 2020

Segmentation client (index_name)

- 10 segments (Ladieswear, Menswear, Sport, Divided...)
- Ladieswear concentre ~41 % des transactions
- Children Sizes 92-140 / 134-170 = enfants

Console interactive — du flux vers le business

Streamlit + Altair, lecture directe Postgres (aucun extract intermédiaire).

Section 1 — Flux temps réel

- KPI pipeline : transactions streamées, fraîcheur
- Caption : validation dbt + GE explicite
- Démo : produce → la console se rafraîchit

Section 2 — Intelligence de la demande

- Lit les marts de référence (31,8 M)
- KPI dynamiques : transactions, clients, articles, % volume
- Filtre par segment → tous les panneaux se recalculent

Mix segment

Catégories

Démographie âge

Palette couleurs

Saisonnalité

Affinité segment × catégorie

Où est la sur-consommation — indice > 1 = signal de recommandation prioritaire.

Méthode & lecture

- Indice = part de la cat. dans le segment / part globale
- Indice = 1 → consommation neutre
- Indice > 1 → sur-consommation (white-space reco)
- Indice < 1 → sous-consommation (à ne pas pousser)
- 8 segments × 8 catégories sur 31,8 M de transactions
- Construit dans pandas à partir de mart_seg_category

Signaux extrêmes (mesurés)

Ladies Accessories	→ Accessories	14,6×
Ladies Accessories	→ Socks & Tights	13,0×
Menswear	→ Socks & Tights	2,7×
Sport	→ Swimwear	2,0×
Children Sizes 92-140	→ Underwear	1,2×

Sortie utile pour Bloc 4 : chaque cellule chargée = piste de reco à prioriser, **par segment**.

Choix d'architecture & arbitrages

Chaque décision est documentée dans le repo (ADR ou commentaire de code).

Kafka KRaft (sans Zookeeper)	Mode officiel 3.8+, déploiement léger, pas de service externe.
Échantillonnage par stride	Étaler 4 500 messages sur 2 ans plutôt qu'un seul jour : courbe saisonnière crédible.
Séparation stream / reference (tag dbt)	DAG rapide (stream) ; marts lourds (reference) build explicite.
Venv Airflow séparé du venv data	Évite les conflits de deps (Polars, dbt 1.x, GE 1.x).
Console Streamlit + Altair	Lecture business directe vs Grafana (sysops). Pas de carte inventée.
Heatmap d'affinité	Une vraie lecture business : sur/sous-indexation vs marché.

Fait pour ce bloc · suite pour les autres

Le scope du Bloc 3 est volontairement bordé. Les briques voisines sont identifiées.

Bloc 3 — livré

- Ingestion Kafka temps réel (KRaft, 3 partitions)
- Consumer → Postgres (idempotent, batch 500)
- Transformations dbt : staging + 6 marts de référence
- Contrôle qualité 2 couches : dbt tests + Great Expectations
- Orchestration Airflow (4 tâches, validation E2E)
- Console interactive Streamlit + heatmap d'affinité
- Code versionné sur GitHub (princebn/miroir)

Hors périmètre (Blocs voisins)

- Bloc 4 — Embeddings CLIP (vector(512))
- Bloc 4 — Reco hybride (collab + content + colorimétrie)
- Bloc 4 — API FastAPI + CI/CD + réentraînement auto
- Bloc 4 — Monitoring drift (Evidently)
- Bloc 1 — Politique de gouvernance & RGPD
- Bloc 1 — Cartographie parties prenantes
- Bloc 1 — Plan d'audit & sensibilisation

CONCLUSION

Un pipeline temps réel industrialisé, des signaux exploitables.

- Code, qualité, orchestration et monitoring tous opérationnels.
- Heatmap d'affinité : sortie directe pour piloter la reco du Bloc 4.
- Architecture reproductible, versionnée, prête à étendre.